

MG-MotionLLM: Research Paper and Technical Implementation Report

Ted Ng Min Teck

ted.ngmt@outlook.com

*A review of MG-MotionLLM (CVPR 2025) [arXiv:2504.02478]
with a technical account of the visualization and Unity streaming pipeline*

Abstract

MG-MotionLLM is a unified motion-language model that performs both motion comprehension (motion→text) and generation (text→motion) across *multiple granularities*, from coarse one-sentence captions to fine-grained, per-body-part, per-time-window “motion scripts.” This report summarizes the model’s architecture (a frozen motion VQ-VAE tokenizer coupled to a T5-based motion-aware language model with a unified motion+text vocabulary) and its two-stage multi-granularity training scheme. We then discuss the choice of an SMPL-based representation over file formats such as BVH/FBX, contrast the simple (coarse) and detailed (fine-grained) captioning tasks, and give a technical account of the implemented inference pipeline: an offline matplotlib renderer and a live Unity WebSocket stream whose receiver reconstructs SMPL bone rotations from convention-free global joint positions.

1. Introduction

Motion comprehension and generation underpin applications across AR/VR, games, and animation. Recent motion-aware large language models (MotionGPT, MotionLLM, M³GPT) unify these directions but model only *coarse-grained* motion↔text, where a few words describe an entire sequence. MG-MotionLLM [1] instead targets **multiple granularities** in one framework: alongside classical text-to-motion and motion captioning, it introduces fine-grained tasks such as motion-to-detailed-text and temporal motion localization. The model is pretrained on **28 motion-relevant tasks** (12 classical coarse + 16 new fine-grained), and the central finding is that these granularities *mutually reinforce* one another, improving even the classical coarse benchmarks.

This report has two purposes: (i) to summarize the paper’s model and design choices, and (ii) to document the accompanying technical pipeline (in particular the visualization and Unity streaming components) as implemented in the released codebase.

2. Model architecture

MG-MotionLLM consists of two components, each frozen after its own training: a motion VQ-VAE that converts between raw motion and discrete tokens, and a T5-based motion-aware language model that aligns motion tokens with text.

2.1 Motion VQ-VAE (tokenizer)

Adopted from T2M-GPT [2], the VQ-VAE has an encoder E , decoder D , and a learnable codebook

$B = \{b_k\}_{k=1}^K$. A T -frame motion $M = [m_1, \dots, m_T]$, $m_t \in \mathbb{R}^{263}$, is encoded and temporally downsampled by rate l (~ 4 frames per token), then each latent z_i is quantized to its nearest codebook entry:

$$c_i = \arg \min_{b_k \in B} \|z_i - b_k\|_2. \quad (1)$$

The decoder reconstructs $\hat{M} = D(\hat{Z})$, and training minimizes reconstruction, embedding, and commitment terms:

$$\mathcal{L} = \|M - \hat{M}\|_2 + \|\text{sg}(Z) - \hat{Z}\|_2 + \beta \|Z - \text{sg}(\hat{Z})\|_2, \quad (2)$$

with $\text{sg}(\cdot)$ the stop-gradient operator. In the codebase (`utils/inference_utils.py`), `vqvae_net.encode()` emits the literal string `<Motion Tokens><id>...</Motion Tokens>`; the codebook size is $K = 1024$.

2.2 Motion-aware language model

The text vocabulary V_t is extended with a motion vocabulary V_m (one token per codebook index) and special tokens V_s : `<Motion Tokens>`/`</Motion Tokens>`, `<SEP>` (separating per-snippet descriptions), and `<Motionless>` (a snippet with no significant movement). With the unified vocabulary $V = \{V_t, V_m, V_s\}$, *every* task becomes sequence-to-sequence: instruction + input tokens $\rightarrow$ output tokens. Following MotionGPT [4], the backbone is **T5**; the released checkpoints are plain **T5ForConditionalGeneration** models decoded greedily (`num_beams=1`, `do_sample=False`).

2.3 Two-stage training

- Granularity-Synergy Pre-training:** all 28 tasks jointly, so coarse tasks bootstrap semantics for fine-grained ones while auxiliary localization/captioning tasks sharpen temporal and detail understanding. (Direct tuning on detailed text alone *hurt* t2m Top-3 retrieval, 77.3% $\rightarrow$ 75.0%: the added detail drowned out the global semantics.)
- Task-Specific Instruction Tuning:** fine-tune the pretrained base per task. Checkpoints follow `*-ft-from-GSPretrained-{small,base,large}`.

3. Simple vs. detailed captioning

The two captioning granularities are distinct tasks with different data, formats, and pipelines.

3.1 Simple caption: Motion-to-Text (m2t)

A single short sentence summarizing the clip (“a person walks forward at a normal pace”). Ground truth

is HumanML3D’s 44,970 sequence-level descriptions (over 14,616 motions, downsampled to 20 fps). Output is short (`max_new_tokens=40`), one caption per clip, rendered as a static title.

3.2 Detailed caption: Motion-to-Detailed-Text (m2dt)

A time-segmented, body-part-level “motion script.” The detailed text comes from FineMotion [6], which re-annotates HumanML3D motions at a fixed 0.5 s granularity. FineMotion provides *Body-Part Movement Snippet Descriptions* (BPMSD), one short description per 0.5 s snippet, and *Body-Part Movement Paragraphs* (BPMP) that concatenate them into a full-sequence narrative; it contains 21,346 human-annotated and 420,968 automatically generated BPMSD across 29,232 motion sequences. The codebase loads these as `BPMSD_auto.json` and `BPMSD_human.json`. The model joins snippets with `<SEP>` (empty ones become `<Motionless>`), produces long output (`max_new_tokens=1536`), and displays it *in sync* with the motion, changing roughly every half second. Crucially, the clip is truncated so that VQ-VAE token boundaries (4 frames each) and snippet boundaries (0.5 s = 10 frames at 20 fps) stay aligned via an LCM unit length, so the text never drifts out of phase with the pose.

4. Why SMPL, and not BVH/FBX?

The paper does not literally argue “SMPL vs. BVH/FBX”; it inherits the HumanML3D [3] 263-d representation derived from SMPL/SMPL-H bodies in AMASS. SMPL (Skinned Multi-Person Linear model) [5] is a parametric 3D human body model from the Max Planck Institute that factorizes a body into shape (β) and pose (θ) parameters and is compatible with standard engines (Maya, Blender, Unity, Unreal); HumanML3D builds on its 22-joint skeleton. Table 1 contrasts why this representation is the right learning substrate while a raw BVH/FBX file is not.

In short, **SMPL is the body model** (a parametric human giving a consistent 22-joint topology and a `recover_from_ric` path back to 3D positions), whereas **BVH/FBX are file/transport formats** carrying rig-specific rotations with no semantic normalization and no text alignment. One converts SMPL→FBX for a DCC tool *at the end*; one does not train on FBX. The repo reflects this: motion lives as `new_joint_vecs/*.npy` (263-d), and the visual/streaming layers call `recover_from_ric(...)` to obtain 22 global joint positions.

4.1 A closer look at BVH

BVH (Biovision Hierarchy), developed by Biovision and supported across Maya, Blender, and MotionBuilder, is a plain-text motion-capture format [7] with two parts. The `HIERARCHY` section declares the skeleton as a tree: a `ROOT` joint and nested `JOINT` blocks, each carrying an `OFFSET` (the bone’s rest-pose displacement from its parent) and a `CHANNELS` list naming its degrees of freedom, typically six for the root (`Xposition Yposition`

`Zposition Zrotation Xrotation Yrotation`) and three rotation channels for every other joint, with leaf bones terminated by an `End Site`. The `MOTION` section then gives `Frames`, a `Frame Time` (seconds per frame, i.e. the inverse of the fps), and one whitespace-separated row of channel values per frame, in the order the channels were declared.

Several properties of this design are exactly what make BVH a poor *learning* substrate, reinforcing Table 1:

- **Euler-angle rotations.** Poses are stored as per-joint Euler angles (in degrees) applied in a channel-specific order, so they suffer gimbal lock and ambiguous interpolation, and their meaning depends on the rotation-order convention, the same kind of convention mismatch that makes streaming rotations to Unity unsafe (see §5.2).
- **Rotation-only joints, fixed bone lengths.** Only the root carries translation; every other joint is rotation-only, with limb lengths frozen by the rest-pose `OFFSETS`. Recovering global joint positions requires forward kinematics down the chain.
- **No standard skeleton.** Joint names, counts, and hierarchy vary by capture system, so two BVH files are generally not comparable without retargeting, unlike SMPL’s single fixed 22-joint topology.
- **Geometry-free.** BVH stores only a skeleton and its animation: no mesh, skin weights, body-shape parameters, or materials. (FBX adds these, but as a heavyweight scene container, not a tokenizable tensor.)

In short, an LLM-friendly motion representation wants a fixed-width, convention-free, shape-normalized tensor; BVH instead offers a variable, convention-dependent, forward-kinematics-bound rotation stream, which is why the HumanML3D/SMPL feature vector, not BVH, is the substrate here.

4.2 A closer look at FBX

FBX (Filmbox) is the de-facto interchange format of the games and VFX industries: Maya, 3ds Max, Blender, Unity, and Unreal all import and export it. It is a *proprietary* format owned by Autodesk (originally Kaydara; Autodesk since 2006) [8], stored as binary or ASCII and read/written through the Autodesk FBX SDK. Unlike BVH, FBX is a full *scene container*: it can hold meshes and geometry, skinning weights, a skeleton, materials and textures, cameras and lights, blendshapes, and multiple animation “takes.”

Given this ubiquity, why does the model not use FBX? Because FBX and SMPL sit at *different levels*, and comparing them is a category error like BVH:

- **FBX is a delivery/authoring format; SMPL is a body model.** They are complementary, not competing. One *exports* a SMPL body to FBX to hand it to an engine; indeed, this report’s own Unity avatar is an FBX import of the SMPL mesh. FBX is used at the delivery end; SMPL is the representation the network actually learns in.
- **Not a tensor.** An FBX file is a heterogeneous, hierarchical scene graph, not a fixed-width numeric

Table 1: Why a SMPL-based feature representation, rather than a BVH or FBX file format.

Aspect	SMPL-based (263-d)	HumanML3D	BVH (Biovision Hierarchy)	FBX (Filmbox)
What it is	Fixed, redundant per-frame feature vector over a canonical 22-joint skeleton (root velocities, local joint positions, 6D joint rotations, velocities, foot-contact flags).		Plain-text mocap format: a skeleton hierarchy plus per-joint Euler-angle rotation tracks (only the root has translation).	Proprietary scene container: meshes, skinning, skeleton, materials, cameras, blendshapes, and animation “takes.”
Skeleton	One universal topology; every clip is directly comparable and learnable.		Arbitrary, rig-specific joint counts/names/offsets; varies by capture system.	Whatever the author rigged; varies per asset. Both need retargeting to compare.
Pose encoding	Compact fixed-width numeric tensor; convention-free joint positions recoverable.		Euler angles in a channel-specific order; gimbal lock, FK needed for positions.	Euler/quaternion tracks inside a hierarchical scene graph; SDK needed to parse.
Body shape	Parametric shape (β) over a canonical mesh.		None; bone lengths frozen in the rest pose, no mesh.	Mesh/skin baked per asset; no canonical shape parameterization.
For an LLM	Already a clean tensor, trivially VQ-VAE-quantizable into discrete tokens.		Variable-length, convention-dependent; no natural fixed-width tokenization.	Heterogeneous scene graph, not a tensor; untokenizable as-is.
Benchmarking	HumanML3D is the standard t2m/m2t benchmark with paired text and metrics.		No standard text pairing or metrics.	No standard text pairing or metrics.

array; there is no natural way to tokenize it for a language model.

- **Rig-specific, shape-free.** The skeleton inside an FBX is whatever the author rigged: joint names, counts, and bind poses vary, so clips are not comparable without retargeting, and there is no canonical body-shape parameterization (SMPL’s β).
- **Proprietary and heavyweight.** Faithful parsing depends on Autodesk’s SDK, and the format carries far more (materials, cameras, lights) than a motion learner needs.

The training substrate therefore stays the compact, fixed, shape-normalized HumanML3D/SMPL feature vector; FBX (and its SMPL avatar) belongs at the visualization/engine end of the pipeline, exactly where the Unity side uses it.

5. Implemented inference pipeline

The released demos wrap the checkpoints with a shared front-end:

```
.npz → normalize → VQ-VAE.encode →
"<Motion Tokens>..." → +instruction →
T5.generate → decode → parse
```

Input priority: explicit `-motion_path`, then dataset `-name`, then every `.npz` under `./input/`, else a random clip from the split. Two output back-ends follow.

5.1 (A) Offline video

`recover_from_riic` turns the 263-d motion into $(T, 22, 3)$ joint positions; `plot_3d_motion` renders a matplotlib 3D-skeleton animation along the SMPL kinematic chain with caption(s) overlaid (static title for m2t, timed captions for m2dt). It saves `<name>.mp4` (or `.gif` without ffmpeg) plus a `<name>.txt` of generated and ground-truth text.

5.2 (B) Live Unity WebSocket stream

A minimal one-way (Python→Unity) JSON-over-WebSocket server (`MotionStreamServer`,

default `ws://0.0.0.0:8765`) emits `start` → per-frame `frame` (`{frame, joints:[x,y,z]×22, rotations:[x,y,z,w]×22, caption}`) → `end`. Key design points:

- **Positions and global rotations.** Each frame carries both: 22 global joint positions and 22 global joint orientations (quaternions). *Local* cont6d rotations cannot be applied to Unity bones directly: HumanML3D/T2M rotate a bone’s offset by the *child’s* accumulated global rotation, Unity’s `Transform.localRotation` by the *parent’s*, so they agree only at rest pose. The `rotations` are therefore each joint’s *global* orientation, recovered from the cont6d channels by forward kinematics and X-mirrored into Unity’s frame; being global they are convention-free (like positions) and additionally carry the per-bone axial twist that positions cannot express (verified to agree with the streamed positions to $<0.04^\circ$).
- **Coordinate fix:** HumanML3D is right-handed Y-up, Unity left-handed Y-up, so X is mirrored (`joints[...][0]*=-1`), preserving heading.
- **Joint order** matches the Unity rig: `Pelvis(0) ... R_Wrist(21)`; hand joints 22/23 stay at bind pose (HumanML3D doesn’t cover them).
- **Static-tail trimming** (`active_length()`) drops the long “stand still” tail so motion and cycling captions end together.
- **Playback** is paced by fps and a `-speed` multiplier; the per-frame caption is whichever 0.5 s snippet is active.

5.3 Why a WebSocket transport?

The sender (the PyTorch model) and the receiver (Unity) almost never run in the same place: inference runs in **Python**, typically on **Linux** with CUDA (here under WSL), while the Unity SMPL renderer runs on

Windows. A WebSocket over TCP is the simplest bridge that crosses this boundary:

- **Cross-platform, cross-process, cross-machine.** The same JSON-over-TCP stream works whether Python and Unity are two processes on one box, a Linux GPU host and a Windows workstation, or two machines on a LAN, with no OS-specific IPC.
- **Language-agnostic and minimal.** The Python side uses the `websockets` library; the Unity side is a tiny hand-rolled C# client. Plain-text JSON frames are trivial to produce and parse on both ends.
- **Real-time push.** A persistent, full-duplex connection lets the server push one pose per frame at the clip's fps with low latency, a far better fit for streaming animation than per-frame HTTP request/response.
- **Decouples compute from rendering.** Heavy GPU inference can stay on a headless server while a separate machine renders the avatar smoothly.

The alternatives are worse fits: dumping intermediate `.npy/.bvh` files is not live; embedding a Python runtime inside Unity 5.6 invites version/ABI conflicts; named pipes and shared memory are same-machine and OS-specific; and gRPC/REST add tooling while still being oriented around request/response rather than continuous push. WebSocket is ubiquitous, firewall-friendly, and crosses Linux↔Windows with zero platform-specific code.

6. The Unity receiver (`smpl_mecanim`)

The Python stream scripts are only the sender; the receiver is a separate Unity 5.6.2f1 project, `smpl_mecanim`, that renders the SMPL body model and drives it live. When idle, each avatar falls back to a Mecanim idle loop.

6.1 Scripts

- `MinimalWebSocketClient.cs`: a hand-rolled RFC-6455 client (Unity 5.6 has no native WebSocket): HTTP Upgrade handshake with Sec-WebSocket-Accept verification, a background read thread, and frames drained on the main thread via `TryDequeue` (Unity APIs are not thread-safe).
- `MotionStreamClient.cs`: the receiver/driver.
- `CaptionDisplay.cs`: the on-screen overlay (legacy OnGUI; the scene has no Canvas).
- `SMPLBlendshapes.ApplyStreamedJoints()` bridges into `SMPLModifyBones.updateBoneAnglesFromJoints()`, the position→rotation retargeter.

6.2 Receive and drive

`MotionStreamClient` self-bootstraps via `RuntimeInitializeOnLoadMethod` (no scene editing) and auto-reconnects every 2s. It parses the stream with SimpleJSON, and applies the pose in `LateUpdate` (after the Animator phase) so a streamed pose wins over the Mecanim idle and is held steady between messages (Python streams ~20–30 fps, Unity renders faster). With interpolation on, it Lerp between consecutive poses over the measured arrival interval, staying smooth and auto-adapting to the sender's `-fps/-speed`.

A dropped connection is treated as `end`, releasing control back to idle. Captions read `caption` or, in compare mode, `caption_m2t+caption_m2dt`, labeled “Simple”/“Detail.”

6.3 The retarget

The receiver supports two retargets. The *position-based* one (`updateBoneAnglesFromJoints`, the default and automatic fallback) rebuilds each bone's rotation from the global joint positions by aiming the bone at its child joint. Over the 22-joint map it captures the bind pose once in avatar-local space, then:

- **Pelvis:** full orientation from the spine direction (up, Pelvis→Spine1) and hip line (right, L_Hip→R_Hip); positioned from `targets[0]`.
- **Torso (spine/neck):** full basis with a blended hip/shoulder “right” line, so twist is reproduced rather than shearing the mesh.
- **Hips (thighs):** twist recovered from the knee→ankle pole so the knee points correctly.
- **Other limbs:** parent-relative swing: inherit the parent's world rotation, then bend only to aim at the next joint (swing-only, no relative twist).

Aiming recovers only swing, so the spine/neck and hips above rely on hardcoded heuristics (a blended hip/shoulder line; a knee-pole sign) to pin the missing twist, and the other limbs stay swing-only. The *rotation-driven* retarget (`updateBoneAnglesFromGlobalRotations`) instead poses the avatar by pure forward kinematics from the streamed global orientations (§5.2): only the pelvis translates, every bone is rotation-only, and children inherit the parent transform at their bind offsets. Each Unity bone i is driven by its primary child's orientation $Q[\text{child}]$ (because Q_j orients the segment *ending* at j , whereas a Unity transform orients the segment *leaving* it), which carries the per-bone axial twist directly from the data and so needs *none* of the hardcoded twist heuristics. It is selected by the `driveByRotation` flag, with the position path as an automatic fallback when a frame lacks rotations.

6.4 Running it

The SMPL package is research-licensed and not committed (only the modified C# scripts are tracked); one downloads the SMPL Unity package, assigns avatar materials, imports idle/walk mocap, and opens in Unity 5.6.x. Then run e.g. `python3 m2t_unity_stream.py -model_name ./m2t-ft-from-GSPretrained-base -name 000000` and press Play; the client auto-connects to `ws://127.0.0.1:8765`.

7. Conclusion

MG-MotionLLM unifies coarse and fine-grained motion comprehension and generation in a single T5-based model over a shared motion+text vocabulary, with a frozen VQ-VAE tokenizer and a two-stage, multi-granularity training scheme in which tasks at different granularities reinforce one another. Its reliance on the SMPL-based HumanML3D representation, rather than rig-specific file formats, is what makes motion

tokenizable and benchmarkable. The accompanying pipeline turns generated motion and text into either offline video or a live Unity stream; the streaming design’s key insight is to transmit convention-free joint *positions* and reconstruct bone rotations on the Unity side by aiming each bone at its child joint, with an optional rotation-driven mode that poses the avatar from the streamed global joint orientations to additionally recover the per-bone axial twist that aiming omits.

Acknowledgments and AI Use

In the interest of transparency, the author discloses that both the technical implementation (the visualization and Unity streaming pipeline built on top of the original MG-MotionLLM release) and the writing of this report were produced with the assistance of an AI coding assistant (Anthropic’s Claude). The AI was used for code implementation, debugging, and drafting; all outputs were reviewed, tested, and verified by the author, who takes full responsibility for the content. The underlying MG-MotionLLM model, datasets, and SMPL body model are the work of their respective authors, as cited.

8. Resources and Links

A categorized list of code, datasets, and background material used or referenced in this report.

8.1 Research paper

- <https://arxiv.org/abs/2504.02478>
MG-MotionLLM: A Unified Framework for Motion Comprehension and Generation across Multiple Granularities (CVPR 2025): the paper summarized in this report.

8.2 Implementation

- <https://github.com/BizhuWu/MG-MotionLLM>
Official MG-MotionLLM repository (this work and the streaming/visualization add-on).
- <https://github.com/EricGuo5513/HumanML3D>
HumanML3D: motion-text dataset (14,616 motions, 44,970 descriptions, 20 fps) and the 263-d motion representation generation.
- <https://github.com/BizhuWu/FineMotion>
FineMotion: fine-grained, per-body-part detailed-caption dataset (BPMSD/BPMP at 0.5 s granularity).
- <https://smpl.is.tue.mpg.de/>
SMPL body model: used for the Unity avatar/rig (the `smpl_mecanim` receiver).

8.3 Further reading and background

- <https://medium.com/meshcapade/the-story-of-f-smpl-3ecea53cb5ed>
Meshcapade: “The Story of SMPL”: accessible background on the SMPL body model.
- <https://me.meshcapade.com>
Meshcapade: SMPL-based avatar and motion tooling.
- <https://research.cs.wisc.edu/graphics/Courses/cs-838-1999/Jeff/BVH.html>
BVH file-format specification: HIERARCHY/MOTION sections, OFFSET/CHANNELS fields, Euler-angle channels.
- <https://www.autodesk.com/products/fbx/overview>

Autodesk FBX: the proprietary interchange/scene format used by Maya, 3ds Max, Blender, Unity, and Unreal.

References

- [1] B. Wu, J. Xie, K. Shen, Z. Kong, J. Ren, R. Bai, R. Qu, and L. Shen. “MG-MotionLLM: A Unified Framework for Motion Comprehension and Generation across Multiple Granularities.” *CVPR*, 2025. arXiv:2504.02478.
- [2] J. Zhang et al. “T2M-GPT: Generating Human Motion from Textual Descriptions with Discrete Representations.” *CVPR*, 2023.
- [3] C. Guo et al. “Generating Diverse and Natural 3D Human Motions from Text.” *CVPR*, 2022.
- [4] B. Jiang et al. “MotionGPT: Human Motion as a Foreign Language.” *NeurIPS*, 2023.
- [5] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. J. Black. “SMPL: A Skinned Multi-Person Linear Model.” *ACM Trans. Graphics (SIGGRAPH Asia)*, 34(6), 2015. <https://smpl.is.tue.mpg.de/>
- [6] B. Wu et al. “FineMotion: A fine-grained, body-part-level human motion description dataset.” <https://github.com/BizhuWu/FineMotion>
- [7] Biovision. “Biovision Hierarchy (BVH) Motion Capture File Format.” Format specification. <https://research.cs.wisc.edu/graphics/Courses/cs-838-1999/Jeff/BVH.html>
- [8] Autodesk. “FBX (Filmbox): 3D Asset Exchange Format and FBX SDK.” Proprietary format (orig. Kaydara; Autodesk since 2006). <https://www.autodesk.com/products/fbx/overview>